Documentación Técnica: Tarea 3 - Asociatividad y Precedencia

Desarrollador: Juan Camilo Lozano Cortés

Lenguaje: Python

1. Preparación del Entorno:

Se utilizó el entorno de Linux con la librería Lark para el análisis sintáctico y pydot junto con el motor Graphviz para la exportación de gráficos. Para este punto, se configuró una gramática extendida que permite diferenciar el comportamiento de crecimiento de los árboles (recursividad) según el operador procesado.

2. Definición de la Gramática y Lógica de Asociatividad:

Code Blame 52 lines (41 loc) · 1.55 KB

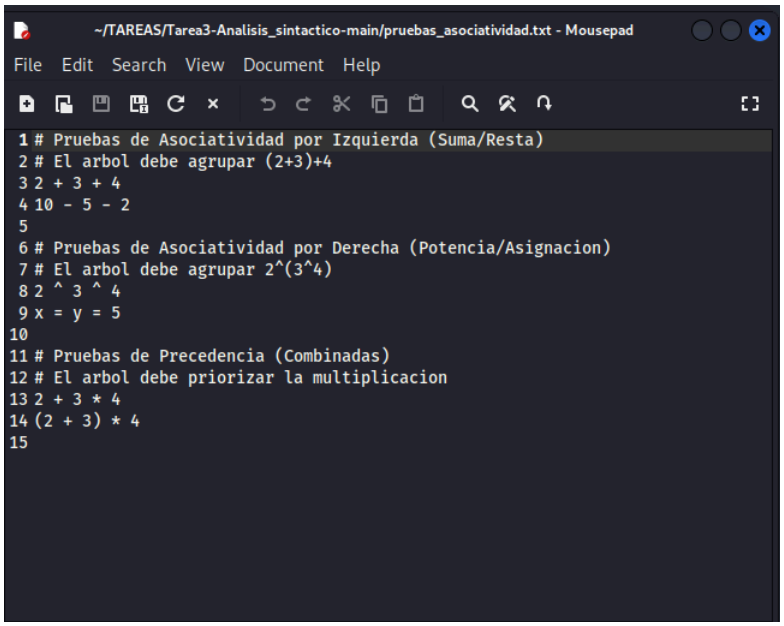
```python
import sys
from lark import Lark, tree

def ejecutar_pruebas(archivo_txt):
    # Gramática completa para asociatividad y precedencia
    grammar = r"""
        ?start: asignacion | expr_suma

        # Asociatividad por Derecha (Asignación)
        asignacion: ID "=" asignacion | ID "=" num

        # Jerarquía de operaciones (Asociatividad y Precedencia)
        ?expr_suma: expr_suma ("+" | "-") term  -> suma_izq  // Izquierda
                  | term

        ?term: term ("*" | "/") factor          -> mul_izq   // Izquierda
             | factor

        ?factor: atom "^" factor                -> pot_der   // Derecha
               | atom

        ?atom: num
             | ID
             | "(" expr_suma ")"

        num: /\d+/
        ID: /[a-z]/
        %import common.WS
        %ignore WS
    """

    parser = Lark(grammar, start='start', parser='earley')

    try:
        with open(archivo_txt, 'r') as f:
            lineas = [l.strip() for l in f if l.strip() and not l.startswith("#")]
    except FileNotFoundError:
        print(f"Error: No se encontró {archivo_txt}")
        return

    for i, cadena in enumerate(lineas):
        try:
            arbol = parser.parse(cadena)
            nombre = f"resultado_asoc_{i+1}.png"
            tree.pydot__tree_to_png(arbol, nombre)
            print(f"Procesada: {cadena} -> {nombre}")
        except Exception as e:
            print(f"Error en '{cadena}': {e}")
```

Se implementó una gramática que maneja tres niveles de jerarquía y dos tipos de asociatividad:

- **Asociatividad por Izquierda:** Implementada en Suma (+) y Resta (-), donde la regla se llama a sí misma en la posición izquierda.
- **Asociatividad por Derecha:** Implementada en Potencia (^) y Asignación (=), donde la regla se llama a sí misma en la posición derecha.

- **Precedencia:** Estructurada en niveles (expr > term > factor > atom), asegurando que las potencias se evalúen antes que las multiplicaciones, y estas antes que las sumas.

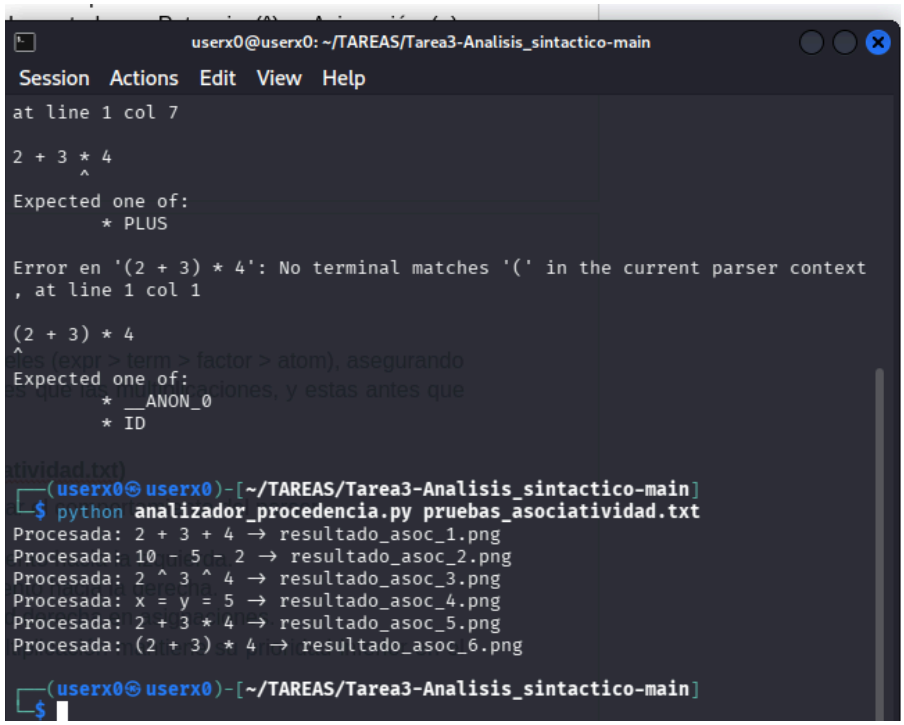
3. Archivos de Prueba (pruebas_asociatividad.txt)



Se diseñaron casos específicos para forzar el comportamiento del parser:

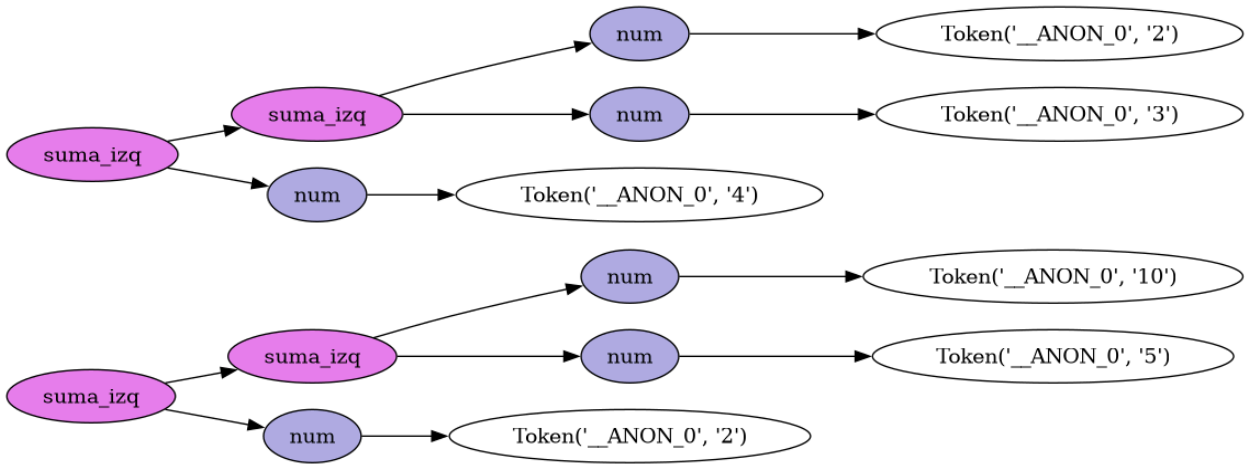
- 2 + 3 + 4: Para observar el crecimiento hacia la izquierda.
- 2 ^ 3 ^ 4: Para observar el crecimiento hacia la derecha.
- x = y = 5: Para validar asociatividad derecha en asignaciones.
- 2 + 3 * 4: Para validar que la multiplicación mantiene su prioridad inferior en el árbol.

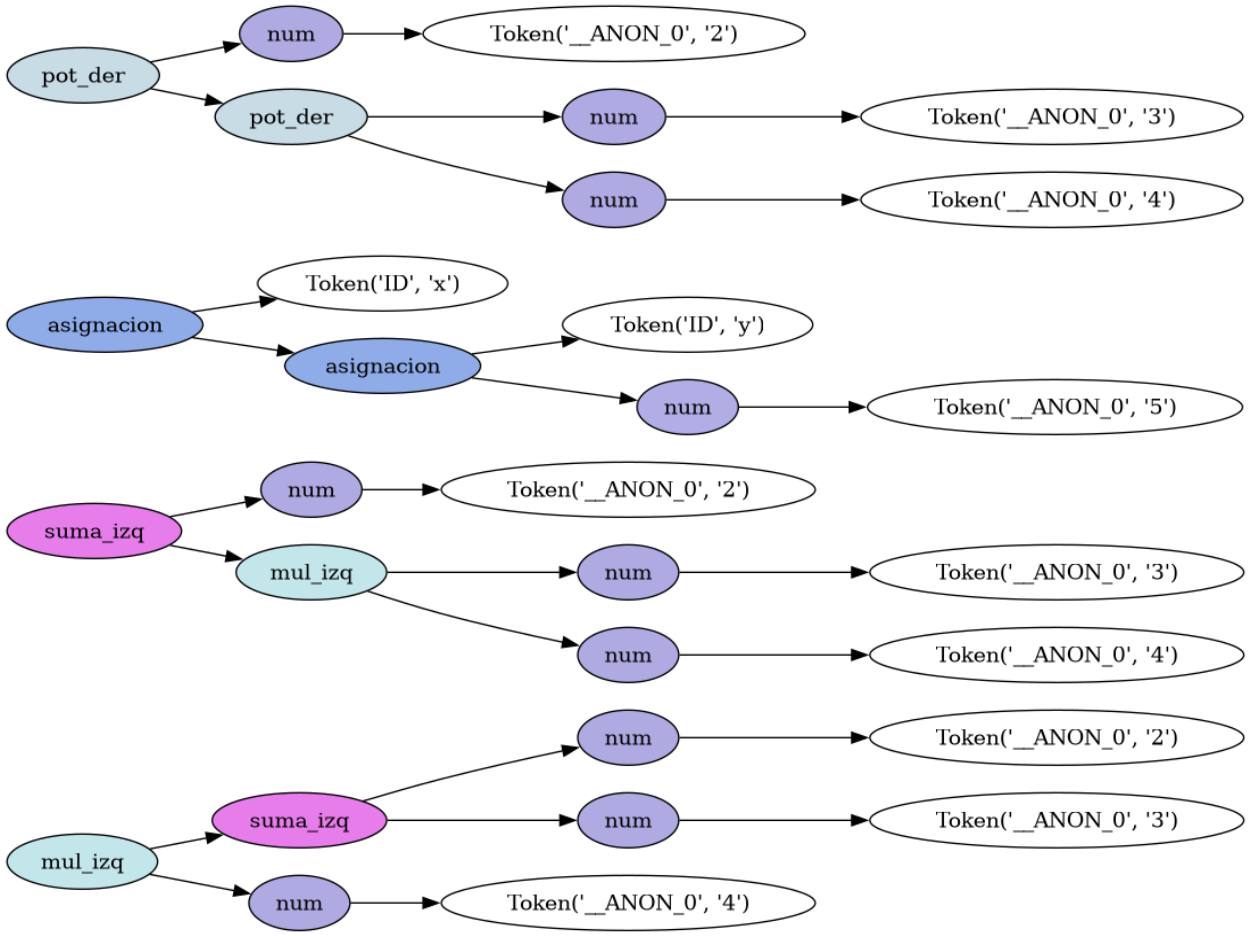
4. Ejecución del Programa:



El script analizador_procedencia.py recorre el archivo de texto, ignora comentarios y espacios en blanco, y genera un archivo PNG por cada expresión. Se utilizó el motor Earley por su capacidad para procesar gramáticas con recursividad mixta (izquierda y derecha) sin ambigüedades.

5. Análisis de Resultados (Evidencia Visual)





Al ejecutar el programa, se generan archivos como resultado_asoc_1.png, resultado_asoc_2.png, etc. Los hallazgos principales son:

- **Caso Suma (Asoc. Izquierda):** En la imagen de 2+3+4, el árbol se expande hacia la izquierda. El compilador agrupa (2+3) primero, y el resultado se convierte en el operando izquierdo de la siguiente suma.
- **Caso Potencia (Asoc. Derecha):** En la imagen de 2^3^4, el árbol se expande hacia la derecha. Esto indica que se evalúa primero (3^4) y ese resultado actúa como el exponente de 2, cumpliendo con la regla matemática de potencias en torre.
- **Caso Asignación:** La cadena x = y = 5 muestra una estructura idéntica a la potencia, confirmando que el valor 5 se asigna primero a y, y el resultado de esa operación se asigna a x.
- **Caso Precedencia Mixta:** En 2 + 3 * 4, se confirma visualmente que el nodo de multiplicación (*) está en un nivel más profundo que la suma (+), garantizando que se ejecute primero en el flujo del compilador.